



ENGENHEIRO DE QUALIDADE DE SOFTWARE

Eduardo Ferreira Gonçalves dos Santos

Análise de Qualidade

Brasília-DF

2026

1. RESUMO

Este Trabalho de Conclusão de Curso apresenta a estratégia de Qualidade de Software aplicada ao e-commerce EBAC Shop, integrando testes manuais e automatizados a um pipeline de Integração Contínua (CI/CD) com o objetivo de garantir a qualidade, estabilidade e confiabilidade da aplicação, validar regras de negócio, detectar defeitos precoces e fornecer evidências consistentes de qualidade. A abordagem de testes contempla as plataformas Web, API REST e Mobile Android, utilizando ferramentas amplamente adotadas no mercado, como Cypress, Supertest, Cucumber, WebdriverIO, Appium e k6. O projeto foi estruturado de forma modular e organizado por User Stories (Histórias de Usuário), integrando-se a pipelines de CI/CD por meio do GitHub Actions, com geração de relatórios e evidências, evidenciando a importância da automação de testes para a garantia da qualidade do software.

2. SUMÁRIO

1. RESUMO	2
2. SUMÁRIO.....	3
3. INTRODUÇÃO	4
4. O PROJETO.....	5
4.1 – ESTRATÉGIA DE TESTE	5
4.2 – CRITÉRIOS DE ACEITAÇÃO	6
[US-001] – Adicionar item ao carrinho (UI – Web)	6
[US-002] – Login na plataforma (UI – Web)	6
[US-003] – API de Cupons	6
[US-004] – Login no aplicativo (Mobile)	7
[US-005] – Cadastrar usuário (Mobile)	7
[US-006] – Login (Performance)	8
[US-007] – Buscar Produto (Performance)	8
4.3 – CASOS DE TESTES	9
Casos de Teste – Adicionar item ao carrinho UI – Web	9
Casos de Teste – Login na plataforma UI – Web	13
Casos de Teste – API de Cupons GET	15
Casos de Teste – API de Cupons POST	16
Casos de Teste – API de Cupons Contrato	18
Caso de Teste – Login no aplicativo Mobile – Android	19
Caso de Teste – Cadastrar usuário no aplicativo Mobile – Android	20
Caso de Teste – Login Performance	21
Caso de Teste – Buscar Produto Performance	21
4.4 – REPOSITÓRIO NO GITHUB.....	22
4.5 – TESTES AUTOMATIZADOS	23
Automação de Interface Web (UI)	23
Automação de API	24
Automação Mobile	25
4.6 – Boas Práticas e Relatórios	26
Relatórios Allure	27
Relatórios de Performance (k6)	27
4.7 – Integração contínua.....	27
Comportamento dos Testes de UI no Ambiente de CI	28
Possíveis Motivos para a Diferença de Comportamento	28
Considerações Importantes	29
4.8 – Testes de Performance	30
Configurações dos Testes de Performance	30
Massa de Dados Utilizada	30
Resultados e Evidências	31
5. CONCLUSÃO	31
6. REFERÊNCIAS BIBLIOGRÁFICAS	33

3. INTRODUÇÃO

A crescente complexidade dos sistemas de software, especialmente em ambientes de comércio eletrônico, tem ampliado a necessidade de práticas eficazes de Qualidade de Software. Aplicações desse tipo exigem alto nível de confiabilidade, desempenho e segurança, uma vez que impactam diretamente a experiência do usuário e os resultados do negócio. Nesse contexto, falhas funcionais, indisponibilidade ou degradação de performance podem gerar prejuízos financeiros e comprometer a credibilidade da plataforma.

A área de testes de software desempenha papel fundamental na mitigação desses riscos, atuando de forma preventiva ao longo do ciclo de desenvolvimento. A adoção de estratégias que combinem testes manuais e automatizados, aliadas a processos de Integração Contínua, possibilita maior rapidez na identificação de defeitos, padronização na execução dos testes e maior confiabilidade nos resultados obtidos. Além disso, a automação contribui para a reprodutibilidade dos testes e para a geração de evidências objetivas que apoiam a tomada de decisão.

Diante desse cenário, este trabalho foi desenvolvido com o propósito de aplicar, de forma prática, conceitos, técnicas e ferramentas de testes de software em um projeto realista de e-commerce. A proposta busca demonstrar como uma estratégia estruturada de testes, integrada a pipelines automatizados, pode contribuir para a melhoria contínua da qualidade do software, alinhando práticas técnicas às demandas do mercado e às boas práticas da engenharia de software.

4. O PROJETO

Para o desenvolvimento deste projeto, são aplicados os conhecimentos adquiridos ao longo do curso com o objetivo de definir e implementar uma estratégia de testes consistente para a validação do e-commerce EBAC Shop, disponível em: <http://lojaebac.ebaconline.art.br/>. As Histórias de Usuário, previamente refinadas, constituem a base para a definição e a cobertura dos cenários de teste, assegurando a verificação sistemática dos requisitos funcionais e não funcionais da aplicação. Dessa forma, o projeto busca garantir que as funcionalidades atendam ao comportamento esperado, bem como que aspectos como desempenho, estabilidade e confiabilidade do sistema sejam devidamente avaliados.

4.1 – ESTRATÉGIA DE TESTE



Figura 1: Mapa Mental - <https://app.xmind.com/>

4.2– CRITÉRIOS DE ACEITAÇÃO

[US-001] – Adicionar item ao carrinho (UI – Web)

Descrição:

- **Como** cliente da EBAC-SHOP
- **Quero** adicionar produtos no carrinho
- **Para** realizar a compra dos itens

CrITÉrios de Aceitação

- O sistema deve impedir a adiç o de mais de 10 unidades de um mesmo produto no carrinho.
- O sistema deve impedir que o valor total do carrinho ultrapasse R\$ 990,00.
- O sistema deve permitir aplicar cupom de desconto de 10% para valores totais entre R\$ 200,00 e R\$ 600,00.
- O sistema deve aplicar automaticamente cupom de desconto de 15% para valores totais acima de R\$ 600,00.

[US-002] – Login na plataforma (UI – Web)

Descri  o:

- **Como** cliente da EBAC-SHOP
- **Quero** fazer o login (autentica  o) na plataforma
- **Para** visualizar meus pedidos

CrITÉrios de Aceita  o

- O sistema deve realizar o login com sucesso quando usu rio e senha forem v lidos.
- O sistema deve exibir mensagem de erro quando o usu rio informar login ou senha incorretos.
- O sistema deve bloquear o login por 15 minutos ap s 3 tentativas consecutivas de senha incorreta

[US-003] – API de Cupons

Descri  o:

- **Como** admin da EBAC-SHOP
- **Quero** criar um servi o de cupom
- **Para** poder listar e cadastrar os cupons

CrITÉrios de Aceita  o

- A API deve permitir listar todos os cupons cadastrados, somente quando a requisic o possuir autentica  o v lida.

- A API deve permitir consultar cupons por ID, retornando os dados correspondentes ao cupom informado.
- A API deve permitir o cadastro de cupons quando todos os campos obrigatórios forem informados corretamente.
- A API deve impedir o cadastro de cupons com código (nome) duplicado.
- A API deve retornar as respostas em conformidade com o contrato definido na documentação do serviço.
- Campos não obrigatórios devem ser tratados como opcionais no cadastro do cupom.

Campos obrigatórios para cadastro do cupom

- code
- amount
- discount_type = fixed_product
- description

[US-004] – Login no aplicativo (Mobile)

Descrição:

- **Como** usuário do aplicativo EBAC-SHOP
- **Quero** realizar login no aplicativo
- **Para** acessar meu perfil e funcionalidades da plataforma

Critérios de Aceitação

- O aplicativo deve permitir que o usuário acesse a tela de login a partir do menu de profile (perfil).
- O aplicativo deve autenticar o usuário (válido) e direcioná-lo para a área de perfil após login bem-sucedido.
- O aplicativo deve exibir as informações do perfil do usuário autenticado após o login no menu de profile (perfil).

[US-005] – Cadastrar usuário (Mobile)

Descrição:

- **Como** novo usuário do aplicativo EBAC-SHOP
- **Quero** realizar meu cadastro no aplicativo
- **Para** acessar as funcionalidades da plataforma

Critérios de Aceitação

- O aplicativo deve permitir que o usuário acesse a opção de novo cadastro a partir do menu profile (perfil).
- O aplicativo deve permitir o cadastro de um novo usuário quando todos os campos obrigatórios forem preenchidos corretamente.
- O aplicativo deve permitir o cadastro quando senha e confirmação de senha forem iguais.

- O aplicativo deve concluir o cadastro e autenticar o usuário após o envio dos dados

Campos obrigatórios para cadastro

- Nome
- Sobrenome
- Telefone
- Email
- Senha
- Confirma senha

[US-006] – Login (Performance)

Descrição:

- **Como** sistema da EBAC-SHOP
- **Quero** suportar múltiplas autenticações de login simultâneas
- **Para** garantir desempenho e estabilidade da plataforma sob carga

Critérios de Aceitação

- O sistema deve suportar múltiplas requisições simultâneas de login.
- O sistema deve manter o tempo de resposta das requisições de autenticação dentro dos limites aceitáveis.
- A taxa de falhas das requisições de login deve permanecer abaixo do limite definido.
- O sistema não deve apresentar indisponibilidade durante o teste de carga.
- O sistema deve manter a integridade do processo de autenticação durante toda a execução do teste.
- O sistema deve gerar relatórios de desempenho ao final da execução do teste.

Valores de referência para o teste

- **Carga de Usuários Simultâneos** – Até 20 usuários simultâneos
- **Duração do Teste** – Aproximadamente 2 minutos e 10 segundos
- **Tempo de Resposta** – Percentil 95 inferior a 5.000 ms
- **Taxa de Falhas** – Inferior a 1% das requisições
- **Taxa de Verificações Funcionais (Checks)** – Igual ou superior a 99%

[US-007] – Buscar Produto (Performance)

Descrição:

- **Como** sistema da EBAC-SHOP
- **Quero** suportar múltiplas requisições simultâneas de busca e visualização de produtos
- **Para** garantir desempenho e estabilidade da página de detalhes do produto sob carga

Critérios de Aceitação

- O sistema deve permitir múltiplos usuários acessando simultaneamente a página de detalhes de produto (PDP).
- As requisições de busca e carregamento da PDP devem retornar status HTTP 200 durante toda a execução do teste.
- O conteúdo HTML da página do produto deve ser retornado corretamente, sem respostas vazias ou incompletas.
- O tempo de resposta das requisições de busca de produto deve permanecer dentro dos limites aceitáveis definidos.
- A taxa de falhas das requisições HTTP deve permanecer abaixo do limite máximo estabelecido.
- A taxa de verificações funcionais (checks) deve permanecer dentro do percentual mínimo aceitável.
- O sistema deve manter estabilidade durante o aumento, sustentação e redução da carga.
- O teste deve gerar relatórios de desempenho para análise dos resultados obtidos.

Valores de referência para o teste

- **Carga de Usuários Simultâneos** – Até 10 usuários simultâneos
- **Duração do Teste** – Aproximadamente 1 minuto e 30 segundos
- **Tempo de Resposta** – Percentil 95 inferior a 5.000 ms
- **Taxa de Falhas** – Inferior a 1% das requisições
- **Taxa de Verificações Funcionais (Checks)** – Igual ou superior a 99%

4.3– CASOS DE TESTES

Casos de Teste – Adicionar item ao carrinho | UI – Web

Premissas

- Plataforma: Web
- Camada: UI
- Ferramenta: Cypress + Cucumber
- Técnicas de teste utilizadas:
 - Partição de Equivalência
 - Análise de Valor Limite
 - Testes de Fronteira Aproximada (Near-Boundary)

CT-CARRINHO-001 Adicionar produto com o limite máximo de quantidade	Objetivo: Validar a adição de produto ao carrinho dentro do limite permitido Técnica: Análise de Valor Limite Pré-condição: Produto disponível no catálogo Passos: 1. Acessar a página de produtos 2. Selecionar um produto 3. Informar quantidade igual a 10 4. Adicionar o produto ao carrinho Resultado Esperado: • Produto adicionado com sucesso • Quantidade aceita pelo sistema • Mensagem de sucesso exibida Tipo de fluxo: Principal Automatizado: Sim
---	--

CT-CARRINHO-002 Impedir adição de produto acima do limite permitido	Objetivo: Validar bloqueio ao exceder o limite máximo de quantidade Técnica: Análise de valor limite Pré-condição: Produto disponível no catálogo Passos: 1. Acessar a página de produtos 2. Selecionar um produto 3. Informar quantidade igual a 11 4. Tentar adicionar o produto ao carrinho Resultado Esperado: • Sistema impede a adição • Mensagem de erro exibida Tipo de fluxo: Negativo Automatizado: Sim
---	---

CT-CARRINHO-003 Finalizar compra com valor total dentro do limite permitido	Objetivo: Validar finalização de compra com valor total até R\$ 990,00 Técnica: Teste de fronteira aproximada (near-boundary) Pré-condição: Produtos disponíveis no catálogo Passos: 1. Adicionar múltiplos produtos ao carrinho 2. Acessar o carrinho 3. Tentar acessar checkout Resultado Esperado: • Sistema permite acessar o checkout • Mensagem de sucesso exibida Tipo de fluxo: Principal Automatizado: Sim
---	--

CT-CARRINHO-004 Impedir finalizar compra com valor acima do limite permitido	Objetivo: Validar bloqueio quando o valor total ultrapassa R\$ 990,00 Técnica: Teste de fronteira aproximada (near-boundary) Pré-condição: Produtos disponíveis no catálogo Passos: 1. Adicionar múltiplos produtos ao carrinho 2. Acessar o carrinho 3. Tentar acessar checkout Resultado Esperado: • Sistema impede o acesso ao checkout • Mensagem de erro exibida Tipo de fluxo: Negativo Automatizado: Sim
--	--

CT-CARRINHO-005 Não aplicar cupom de 10% para valor abaixo de R\$ 200	Objetivo: Validar rejeição do cupom de 10% quando o valor total do carrinho for inferior ao mínimo permitido Técnica: Partição de equivalência (valor inválido – abaixo do limite inferior) Pré-condição: Carrinho com valor total inferior a R\$ 200,00 Passos: 1. Adicionar produtos ao carrinho com valor total inferior a R\$ 200,00 2. Acessar o carrinho 3. Aplicar cupom “techugo10” Resultado Esperado: • Sistema não aplica o desconto • Mensagem de erro exibida Tipo de fluxo: Negativo Automatizado: Sim
---	---

CT-CARRINHO-006 Aplicar cupom de 10% dentro da faixa válida	Objetivo: Validar aplicação de cupom de desconto de 10% Técnica: Partição de equivalência Pré-condição: Carrinho com valor entre R\$ 200,00 e R\$ 600,00 Passos: 1. Adicionar múltiplos produtos ao carrinho 2. Acessar o carrinho 3. Aplicar cupom “techugo10” Resultado Esperado: • Desconto de 10% aplicado corretamente • Mensagem de sucesso exibida Tipo de fluxo: Principal Automatizado: Sim
---	---

CT-CARRINHO-007 Não aplicar cupom de 10% para valor acima de R\$ 600	Objetivo: Validar rejeição do cupom de 10% quando o valor total do carrinho for superior ao máximo permitido Técnica: Partição de equivalência (valor inválido – acima do limite superior) Pré-condição: Carrinho com valor total superior a R\$ 600,00 Passos: 1. Adicionar produtos ao carrinho com valor total superior a R\$ 600,00 2. Acessar o carrinho 3. Aplicar cupom “techugo10” Resultado Esperado: • Sistema não aplica o desconto • Mensagem de erro exibida Tipo de fluxo: Negativo Automatizado: Sim
--	--

CT-CARRINHO-008 Não aplicar cupom de 15% quando valor for igual ou inferior a R\$ 600,00	Objetivo: Validar rejeição do cupom de 15% fora da faixa válida Técnica: Partição de equivalência Pré-condição: Carrinho com valor igual ou inferior a R\$ 600,00 Passos: 1. Adicionar produtos ao carrinho com valor igual ou inferior a R\$ 600,00 2. Acessar o carrinho 3. Aplicar cupom “techugo15” Resultado Esperado: • Sistema não aplica o desconto • Mensagem de erro exibida Tipo de fluxo: Negativo Automatizado: Sim
--	---

CT-CARRINHO-009 Aplicar cupom de 15% quando valor total for superior a R\$ 600,00	Objetivo: Validar aplicação de cupom de 15% Técnica: Partição de equivalência Pré-condição: Carrinho com valor superior a R\$ 600,00 Passos: 1. Adicionar produtos ao carrinho com valor total superior a R\$ 600,00 2. Acessar o carrinho 3. Aplicar cupom “techugo15” Resultado Esperado: • Desconto de 15% aplicado corretamente • Mensagem de sucesso exibida Tipo de fluxo: Principal Automatizado: Sim
---	---

Casos de Teste – Login na plataforma | UI – Web

Premissas

- Plataforma: Web
- Camada: UI
- Ferramenta: Cypress + Cucumber
- Técnica(s) aplicada(s):
 - Partição de Equivalência
 - Tabela de Decisão
 - Análise de Valor Limite (tentativas de login)

CT-LOGIN-001 Login com usuário cadastrado	Objetivo: Validar login com credenciais válidas Técnica: Partição de equivalência (dados válidos) Pré-condição: Usuário cadastrado Passos: 1. Acessar a tela de login 2. Informar usuário válido 3. Informar senha válida 4. Submeter o login Resultado Esperado: • Login realizado com sucesso • Usuário autenticado na plataforma Tipo de fluxo: Principal Automatizado: Sim
CT-LOGIN-002 Login com usuário inválido	Objetivo: Validar comportamento do sistema para usuário inexistente Técnica: Partição de equivalência (dados inválidos) Pré-condição: Usuário não cadastrado Passos: 1. Acessar a tela de login 2. Informar usuário inválido 3. Informar senha válida 4. Submeter o login Resultado Esperado: • Sistema exibe mensagem de erro • Login não realizado Tipo de fluxo: Negativo Automatizado: Sim

CT-LOGIN-003 Login com senha inválida (Fluxo Negativo)	Objetivo: Validar erro de autenticação para senha incorreta Técnica: Partição de equivalência (senha inválida) Pré-condição: Usuário válido cadastrado Passos: 1. Acessar a tela de login 2. Informar usuário válido 3. Informar senha inválida 4. Submeter o login Resultado Esperado: • Sistema exibe mensagem de erro • Login não realizado Tipo de fluxo: Negativo Automatizado: Sim
--	--

CT-LOGIN-004 Não bloquear conta com menos de 3 tentativas inválidas	Objetivo: Validar regra de bloqueio apenas após 3 tentativas Técnica: Análise de valor limite Pré-condição: Usuário ativo Passos: 1. Acessar a tela de login 2. Realizar 2 tentativas de login com senha inválida 3. Tentar realizar nova tentativa de login Resultado Esperado: • Sistema permite nova tentativa de login • Conta não é bloqueada Tipo de fluxo: Alternativo Automatizado: Sim
---	--

CT-LOGIN-005 Bloquear conta após 3 tentativas inválidas	Objetivo: Validar bloqueio de conta após exceder o limite permitido Técnica: Análise de valor limite Pré-condição: Usuário ativo Passos: 1. Acessar a tela de login 2. Realizar 3 tentativas de login com senha inválida 3. Realizar tentativa de login com credenciais válidas Resultado Esperado: • Sistema bloqueia o login • Mensagem informando bloqueio por 15 minutos é exibida Tipo de fluxo: Negativo Automatizado: Sim
---	---

Casos de Teste – API de Cupons | GET

Premissas

- Plataforma: API REST
- Método: GET
- Endpoint: /wp-json/wc/v3/coupons
- Ferramenta: Supertest + Cucumber
- Técnicas de teste utilizadas:
 - Partição de Equivalência
 - Testes Negativos (Autenticação)

CT-API-CUPOM-GET-001 Listar todos os cupons com autenticação válida	Objetivo: Validar a listagem de cupons quando a requisição possui autenticação válida Técnica: Partição de equivalência (requisição válida) Pré-condição: Admin com credenciais válidas Passos: 1. Realizar requisição GET para listagem de cupons com autenticação válida Resultado Esperado: • API retorna status HTTP 200 • Lista de cupons retornada com sucesso Tipo de fluxo: Principal Automatizado: Sim
CT-API-CUPOM-GET-002 Buscar cupom por ID com autenticação válida	Objetivo: Validar consulta de cupom específico por ID Técnica: Partição de equivalência (ID válido) Pré-condição: 1. Admin autenticado 2. Cupom previamente cadastrado Passos: 1. Realizar requisição GET para consulta de cupom por ID com autenticação válida Resultado Esperado: • API retorna status HTTP 200 • Dados do cupom consultado retornados corretamente Tipo de fluxo: Principal Automatizado: Sim

CT-API-CUPOM-GET-003 Bloquear listagem de cupons sem autenticação	Objetivo: Validar bloqueio de acesso quando a requisição não possui autenticação Técnica: Partição de equivalência (requisição inválida – sem autenticação) Pré-condição: Nenhuma Passos: 1. Realizar requisição GET para listagem de cupons sem autenticação Resultado Esperado: • API retorna status HTTP 401 ou 403 • Acesso à listagem de cupons é bloqueado Tipo de fluxo: Negativo Automatizado: Sim
--	---

CT-API-CUPOM-GET-004 Bloquear listagem de cupons com autenticação inválida	Objetivo: Validar bloqueio de acesso quando a autenticação é inválida Técnica: Partição de equivalência (requisição inválida – autenticação incorreta) Pré-condição: Nenhuma Passos: 1. Realizar requisição GET para listagem de cupons com autenticação inválida Resultado Esperado: • API retorna status HTTP 401 ou 403 • Acesso à listagem de cupons é negado Tipo de fluxo: Negativo Automatizado: Sim
---	--

Casos de Teste – API de Cupons | POST

Premissas

- Plataforma: API REST
- Método: POST
- Endpoint: /wp-json/wc/v3/coupons
- Ferramenta: Supertest + Cucumber
- Técnicas de teste utilizadas:
 - Partição de Equivalência
 - Testes Negativos
 - Tabela de Decisão (dados obrigatórios)

CT-API-CUPOM- POST-001 Cadastrar cupom com sucesso	Objetivo: Validar o cadastro de um novo cupom com dados válidos Técnica: Partição de equivalência (dados válidos) Pré-condição: Admin com autenticação válida Passos: 1. Realizar requisição POST para cadastro de cupom com dados válidos e autenticação válida Resultado Esperado: • API retorna status HTTP 201 ou 200 • Cupom criado com sucesso Tipo de fluxo: Principal Automatizado: Sim
---	--

CT-API-CUPOM- POST-002 Bloquear cadastro de cupom com autenticação inválida	Objetivo: Validar bloqueio de cadastro quando a autenticação é inválida Técnica: Partição de equivalência (requisição inválida – autenticação) Pré-condição: Nenhuma Passos: 1. Realizar requisição POST para cadastro de cupom com autenticação inválida Resultado Esperado: • API retorna status HTTP 401 ou 403 • Cadastro de cupom bloqueado Tipo de fluxo: Negativo Automatizado: Sim
--	---

CT-API-CUPOM- POST-003 Não permitir cadastro de cupom com nome duplicado	Objetivo: Validar rejeição de cadastro de cupom com código já existente Técnica: Partição de equivalência (dado inválido – duplicidade) Pré-condição: • Admin autenticado • Cupom previamente cadastrado com o mesmo código (nome) Passos: 1. Realizar requisição POST para cadastro de cupom utilizando código já existente Resultado Esperado: • API retorna status HTTP 400
---	---

	Mensagem informando que o nome do cupom já existe Tipo de fluxo: Negativo Automatizado: Sim
--	---

CT-API-CUPOM-POST-004 Não permitir cadastro de cupom com dados obrigatórios inválidos	Objetivo: Validar erro de validação ao cadastrar cupom com dados obrigatórios inválidos Técnica: Tabela de decisão Pré-condição: Admin autenticado Passos: - Realizar requisição POST para cadastro de cupom informando dados inválidos conforme combinações definidas Resultado Esperado: - API retorna status HTTP 400 - Erro de validação no cadastro do cupom Tipo de fluxo: Negativo Automatizado: Sim
--	--

Tabela de Decisão – Dados Inválidos (CT-API-CUPOM-POST-004)

Caso	Code	Amount	Discount Type	Description	Resultado Esperado
1	Ausente	Válido	Válido	Válido	Erro de validação
2	Válido	Ausente	Válido	Válido	Erro de validação
3	Válido	Válido	Inválido	Válido	Erro de validação
4	Válido	Válido	Válido	Ausente	Erro de validação

Casos de Teste – API de Cupons | Contrato

Premissas

- Plataforma: API REST
- Métodos: GET, POST
- Ferramenta: Supertest + Cucumber
- Tipo de teste: Teste de Contrato
- Técnica de teste:
 - Validação de contrato (schema)
 - Partição de equivalência (resposta válida)

CT-API-CUPOM-CONTRATO-001 Validar contrato do cupom retornado no GET	Objetivo: Garantir que a estrutura do cupom retornado na listagem esteja conforme o contrato definido Técnica: Validação de contrato (schema) Pré-condição: Admin autenticado na API Passos: 1. Realizar requisição GET para listagem de cupons com autenticação válida Resultado Esperado: • API retorna status HTTP 200 • Cada cupom retornado atende ao contrato definido (campos obrigatórios, tipos e estrutura) Tipo de fluxo: Principal Automatizado: Sim
--	---

CT-API-CUPOM-CONTRATO-002 Validar contrato do cupom retornado no POST	Objetivo: Garantir que a estrutura do cupom retornado após o cadastro esteja conforme o contrato definido Técnica: Validação de contrato (schema) Pré-condição: Admin autenticado na API Passos: 1. Realizar requisição POST para cadastro de um novo cupom com dados válidos Resultado Esperado: • API retorna status HTTP 201 ou 200 • Cupom retornado atende ao contrato definido (campos obrigatórios, tipos e estrutura) Tipo de fluxo: Principal Automatizado: Sim
---	---

Caso de Teste – Login no aplicativo | Mobile – Android

Premissas

- Plataforma: Mobile
- Sistema Operacional: Android
- Ferramenta: WebdriverIO + Appium
- Tipo de teste: Funcional
- Escopo: Login com credenciais válidas

CT-MOBILE-LOGIN-001 Realizar login com credenciais válidas	Objetivo: Validar que o usuário consegue realizar login com credenciais válidas no aplicativo. Técnica: Partição de equivalência (dados válidos) Pré-condição: Usuário previamente cadastrado Passos: 1. Acessar o aplicativo 2. Acessar o menu de perfil (profile) 3. Informar e-mail válido 4. Informar senha válida 5. Submeter o login Resultado Esperado: • Login realizado com sucesso • Usuário autenticado • Nome do usuário exibido na área de perfil Tipo de fluxo: Principal Automatizado: Sim
--	---

Caso de Teste – Cadastrar usuário no aplicativo | Mobile – Android

Premissas

- Plataforma: Mobile
- Sistema Operacional: Android
- Ferramenta: WebdriverIO + Appium
- Tipo de teste: Funcional
- Escopo: Cadastro de novo usuário com dados válidos

CT-MOBILE-CADASTRO-001 Cadastrar novo usuário com dados válidos	Objetivo: Validar o cadastro de um novo usuário no aplicativo com dados válidos Técnica: Partição de equivalência (dados válidos) Pré-condição: Aplicativo em funcionamento e acesso à área de cadastro Passos: 1. Acessar o aplicativo 2. Acessar o menu de perfil 3. Selecionar a opção “Sign up” (Inscrever-se) 4. Preencher nome, sobrenome, telefone, e-mail e senha 5. Confirmar a senha 6. Submeter o cadastro Resultado Esperado: • Usuário cadastrado com sucesso • Usuário autenticado automaticamente • E-mail do usuário exibido na área de perfil Tipo de fluxo: Principal Automatizado: Sim
---	--

Caso de Teste – Login | Performance

Premissas

- Tipo de teste: Não funcional – Performance
- Ferramenta: k6
- Executor: ramping-vus
- Escopo: Autenticação de usuários via interface Web (WooCommerce)

CT-PERF-LOGIN-001 Avaliar desempenho do login sob carga simultânea	Objetivo: Validar o desempenho do processo de login com múltiplos usuários simultâneos. Técnica: Teste de carga (Load Test) Pré-condição: Usuários válidos previamente cadastrados no sistema. Passos: 1. Executar teste de carga com rampa progressiva de usuários 2. Realizar requisições GET da página de login para obtenção de nonce (token temporário) 3. Realizar requisições POST de autenticação com credenciais válidas Resultado Esperado: • Página de login retorna status HTTP 200 • Requisição POST de login retorna status HTTP 200 ou 302 • Percentil 95 do tempo de resposta inferior a 5 segundos • Taxa de falhas inferior a 1% • Taxa de checks igual ou superior a 99% Tipo de fluxo: Principal Automatizado: Sim
--	--

Caso de Teste – Buscar Produto | Performance

Premissas

- Tipo de teste: Não funcional – Performance
- Ferramenta: k6
- Executor: ramping-vus
- Escopo: Acesso simultâneo à página de detalhes do produto (PDP)

CT-PERF- PRODUTO-001 Avaliar desempenho da página de produto sob carga simultânea	Objetivo: Validar o desempenho da página de detalhes do produto com múltiplos usuários acessando simultaneamente. Técnica: Teste de carga (Load Test) Pré-condição: Produto disponível no catálogo. Passos: 1. Executar teste de carga com rampa progressiva de usuários 2. Realizar requisições GET para a página de detalhes do produto (PDP) 3. Simular tempo de navegação do usuário (think time) Resultado Esperado: • Página do produto retorna status HTTP 200 • Conteúdo HTML retornado corretamente (não vazio) • Página contém identificador esperado do produto • Percentil 95 do tempo de resposta inferior a 5 segundos • Taxa de falhas inferior a 1% • Taxa de checks igual ou superior a 99% Tipo de fluxo: Principal Automatizado: Sim
--	--

4.4 – REPOSITÓRIO NO GITHUB

Para o desenvolvimento, versionamento e armazenamento dos artefatos produzidos neste Trabalho de Conclusão de Curso (TCC), foi criado um repositório no GitHub denominado TCC-EBAC-QE, disponível pelo no link: <https://github.com/Eduferr/tcc-ebac-qe.git>

O repositório foi mantido público durante todo o período de avaliação pelos tutores, conforme as orientações da EBAC, possibilitando o acesso aos códigos-fonte e às evidências das automações de testes desenvolvidas ao longo do projeto.

Neste repositório encontram-se disponíveis tanto este documento do TCC quanto todos os códigos-fonte das automações de testes implementadas, abrangendo as camadas de Interface Web (UI), API, Mobile e Performance.

A estrutura do repositório foi organizada em pastas, seguindo boas práticas de versionamento, organização e manutenção de projetos de Quality Assurance.

4.5– TESTES AUTOMATIZADOS

Automação de Interface Web (UI)

Para a automação dos testes de Interface Web, foi desenvolvido um projeto utilizando a linguagem JavaScript em conjunto com o framework Cypress, integrado ao Cucumber para escrita dos cenários em BDD. Essa escolha foi baseada em critérios técnicos, facilidade de manutenção e aderência às boas práticas de automação de testes.

Justificativa da Ferramenta e Linguagem

A escolha do Cypress com JavaScript foi realizada após análise comparativa com outras ferramentas amplamente utilizadas no mercado, conforme descrito a seguir:

- **Cypress + JavaScript**
 - Execução rápida e estável em aplicações web modernas
 - Excelente suporte à automação de testes end-to-end
 - Facilidade de escrita e leitura dos testes
 - Integração nativa com ferramentas de CI/CD
 - Boa integração com Cucumber para BDD
- **Selenium + Java**
 - Ferramenta madura e amplamente difundida
 - Maior complexidade de configuração
 - Execução mais lenta quando comparada ao Cypress
 - Necessidade de maior controle de sincronização
- **Playwright + TypeScript**
 - Framework moderno e robusto
 - Suporte multiplataforma (Chromium, Firefox, WebKit)
 - Curva de aprendizado maior
 - Comunidade e ecossistema ainda em crescimento quando comparados ao Cypress

Diante dessa análise, o **Cypress com JavaScript** foi considerado a opção mais adequada para o escopo do projeto, equilibrando produtividade, estabilidade, facilidade de manutenção e curva de aprendizado.

Estrutura e Organização

Foi criada uma pasta denominada UI, destinada exclusivamente aos testes automatizados de interface web. Os testes foram organizados por histórias de usuário e funcionalidades, facilitando a rastreabilidade e a manutenção.

Padrão de Teste Utilizado (Testing Pattern)

Nos testes de UI foi aplicado o **Page Object Model (POM)**, promovendo:

- Separação entre lógica de teste e elementos de interface
- Reutilização de código
- Facilidade de manutenção em caso de alterações na interface
- Maior legibilidade dos testes

Automação de API

Para a automação de testes de API, foi criada uma pasta denominada API, contendo os testes automatizados da API de Cupons, utilizando JavaScript com Supertest e cenários em BDD com Cucumber. O escopo inclui testes funcionais (GET/POST), testes negativos de autenticação e validação de contrato (schema).

Justificativa da Ferramenta e Linguagem (API)

A escolha por Supertest + JavaScript (com Cucumber) foi definida após comparação com alternativas comuns:

- **Supertest + JavaScript**
 - Integração direta com Node.js, simples para validar status, headers e body.

- Ótimo encaixe para pipelines CI/CD e projeto full JS (UI + API + performance).
- Facilidade para estruturar serviços, helpers e reutilização de requisições.
- **Postman/Newman**
 - Excelente para exploração e coleções reutilizáveis.
 - Pode ser limitado para testes mais “código-first” (reuso avançado, asserts customizadas complexas).
 - Integração com BDD e organização por camadas tende a ser menos natural do que em código.
- **RestAssured + Java**
 - Muito robusto e expressivo.
 - Exige stack Java dedicada e aumenta a heterogeneidade tecnológica do projeto.
 - Curva de configuração/estruturação maior para um projeto já padronizado em JS.

Assim, Supertest + JavaScript foi escolhido por manter coerência tecnológica com o projeto, facilitar manutenção e permitir automação escalável com boa legibilidade.

Validação de Contratos (API)

Foram implementados testes de contrato para os retornos de GET e POST de cupons, garantindo que a estrutura esperada (schema) seja respeitada, reduzindo riscos de quebra para consumidores da API.

Automação Mobile

Para a automação mobile, foi criada a pasta Mobile, contendo testes automatizados para Android, utilizando JavaScript com WebdriverIO e Appium, aplicados às funcionalidades automatizadas no aplicativo disponibilizado pela EBAC.

Justificativa da Ferramenta e Linguagem (Mobile)

A escolha por WebDriverIO + Appium + JavaScript foi definida após comparação com as alternativas:

- **WebDriverIO + Appium + JavaScript**
 - Permite automação cross-platform via Appium, mantendo o projeto em JS.
 - Boa integração com padrões (Page Objects), relatórios e execução local.
 - Coerência com as demais camadas (UI, API e performance também em JS).
- **Appium + Java (JUnit/TestNG)**
 - Robusto e amplamente utilizado.
 - Aumenta a diversidade tecnológica e esforço de manutenção (stack paralela em Java).
 - Para o projeto, manter tudo em JS foi mais eficiente.
- **Espresso (Android) / XCUITest (iOS)**
 - São frameworks nativos (alta performance e estabilidade).
 - Exigem implementação separada por plataforma e conhecimento específico.
 - Para um projeto acadêmico com foco em estratégia multicamada, Appium foi mais adequado por unificar a automação.

Assim, WebDriverIO + Appium + JavaScript foi adotado por equilibrar produtividade, integração com o restante do projeto e organização por padrões.

4.6– Boas Práticas e Relatórios

Durante a implementação foram observadas boas práticas de organização e manutenibilidade, com separação por camada (UI/API/Mobile/Performance), reutilização de componentes e rastreabilidade por histórias de usuário.

Quanto às evidências de execução dos testes automatizados, os resultados são armazenados de forma organizada no repositório, separados por camada de teste, conforme descrito a seguir:

Relatórios Allure

Os testes automatizados de Interface Web (UI), API e Mobile geram evidências no formato compatível com o Allure Report. Os arquivos de resultados brutos são armazenados no diretório:

```
allure/
├── ui-tests/ui-results
├── api-tests/api-results
└── mobile-tests/mobile-results
```

A partir desses arquivos, são gerados os relatórios consolidados do Allure, disponibilizados nas respectivas pastas de relatório (*ui-report*, *api-report* e *mobile-report*), permitindo a análise detalhada das execuções, cenários, passos e evidências de falhas.

Relatórios de Performance (k6)

Os testes de performance são executados com a ferramenta k6, que gera relatórios em formatos HTML e JSON. Essas evidências ficam armazenadas no diretório:

```
performance/reports/
├── login
│   ├── login-report.html
│   └── login-summary.json
└── produto
    ├── produto-report.html
    └── produto-summary.json
```

Esses relatórios apresentam métricas de desempenho, como tempo de resposta, taxa de erros e percentis, servindo como evidência objetiva da execução dos testes não funcionais.

4.7 – Integração contínua

A execução dos testes automatizados deste projeto foi integrada a um processo de Integração Contínua (CI) utilizando o GitHub Actions.

O pipeline de CI é acionado automaticamente a cada envio de alterações ao repositório (push) ou por execução manual, realizando a configuração do ambiente, a instalação das dependências, a execução dos testes automatizados e a geração das evidências de teste.

No contexto deste trabalho, foram integrados ao CI os testes automatizados das seguintes camadas:

- Interface Web (UI);
- API;
- Performance.

Os resultados das execuções são armazenados como evidências, permitindo a análise dos testes realizados em ambiente controlado e padronizado.

Comportamento dos Testes de UI no Ambiente de CI

Durante a execução dos testes automatizados de Interface Web (UI) no ambiente de Integração Contínua (GitHub Actions), foi identificado que alguns testes relacionados à funcionalidade de Carrinho de Compras, que apresentaram sucesso no ambiente local, falharam quando executados no ambiente de CI.

Esse comportamento evidencia uma diferença entre o ambiente local de desenvolvimento e o ambiente de execução em CI, cenário comum em projetos de automação de testes.

Possíveis Motivos para a Diferença de Comportamento

Os principais fatores técnicos que podem justificar esse comportamento incluem:

- **Diferenças de desempenho entre ambientes** – O ambiente do GitHub Actions possui recursos computacionais compartilhados, o que pode impactar o tempo de carregamento das páginas e provocar falhas em testes a tempo de resposta.
- **Questões de sincronização e espera (timing)** – Testes que dependem implicitamente do tempo de renderização da interface podem falhar em

ambientes mais lentos, indicando a necessidade de estratégias mais robustas de sincronização (ex.: waits explícitos e validações condicionais).

- **Execução em modo headless** – No CI, os testes de UI são executados em modo **headless**, o que pode gerar comportamentos distintos em relação à execução local com navegador visível.
- **Instabilidade da aplicação alvo** – Por se tratar de um ambiente público e compartilhado, a aplicação sob teste pode apresentar instabilidades momentâneas que afetam a confiabilidade dos testes automatizados.

Considerações Importantes

Mesmo com essas ocorrências, os testes de UI:

- Foram mantidos no pipeline de CI como forma de identificar possíveis fragilidades;
- Serviram como instrumento de diagnóstico para melhorias futuras na estabilidade dos testes;
- Demonstram, no contexto do TCC, a importância da Integração Contínua para revelar problemas que não são percebidos apenas em execuções locais.

Esse cenário reforça a necessidade de ajustes contínuos na automação, bem como a adoção de boas práticas para tornar os testes mais resilientes a variações de ambiente.

Quanto aos testes automatizados Mobile, não foram integrados ao pipeline de Integração Contínua neste projeto, devido à instabilidade observada durante a execução local dos testes, principalmente relacionada a:

- Elementos de interface que não eram carregados corretamente na tela;
- Comportamentos inconsistentes do aplicativo em ambiente de teste;
- Impacto direto na confiabilidade e reprodutibilidade das execuções automatizadas.

Dessa forma, optou-se por manter os testes Mobile restritos à execução local, garantindo maior controle do ambiente e evitando falsos negativos no pipeline de CI.

4.8 – Testes de Performance

Os testes de performance foram implementados utilizando a ferramenta k6, com o objetivo de avaliar o comportamento da aplicação sob carga simultânea de usuários.

Foram automatizados dois casos de teste de performance, contemplando funcionalidades críticas do sistema:

- US-006 – Login (Performance);
- US-007 – Busca e visualização de produto (Performance).

Esses testes permitem analisar métricas relacionadas à estabilidade, tempo de resposta e taxa de falhas da aplicação em cenários de uso concorrente.

Configurações dos Testes de Performance

As configurações adotadas nos testes de performance seguem o padrão abaixo:

- O limite de usuários virtuais variou conforme o cenário testado, sendo de até 20 VUs para autenticação e até 10 VUs para busca de produtos
- Tempo total de execução: aproximadamente 2 minutos
- Ramp-up: 20 segundos para aumento gradual da carga
- Executor: ramping-vus

Massa de Dados Utilizada

Para o teste de autenticação, foi utilizada uma massa de dados composta por usuários e senhas válidos previamente cadastrados no sistema:

- user1_ebac / psw!ebac@test
- user2_ebac / psw!ebac@test
- user3_ebac / psw!ebac@test

- user4_ebac / psw!ebac@test
- user5_ebac / psw!ebac@test

Essa abordagem possibilitou simular acessos simultâneos realistas, garantindo maior aderência dos resultados às condições reais de uso do sistema.

Resultados e Evidências

Os testes de performance geraram relatórios nos formatos **HTML** e **JSON**, contendo métricas como:

- Tempo de resposta (percentis);
- Taxa de falhas das requisições;
- Taxa de sucesso das verificações (checks).

Essas evidências estão armazenadas no repositório do projeto e complementam a estratégia de testes funcionais adotada nas demais camadas.

5. CONCLUSÃO

A realização deste Trabalho de Conclusão de Curso proporcionou uma experiência prática e aprofundada no planejamento, implementação e execução de uma estratégia completa de testes de software, abrangendo diferentes camadas da aplicação. Ao longo do projeto, foi possível aplicar de forma integrada os conhecimentos adquiridos durante o curso, consolidando conceitos fundamentais de Quality Assurance, automação de testes e boas práticas de engenharia de software.

A definição da estratégia de testes, estruturada a partir de histórias de usuário, critérios de aceitação e casos de teste, permitiu compreender a importância da rastreabilidade entre requisitos, testes e evidências. A utilização de técnicas formais de teste, como partição de equivalência, análise de valor limite, tabela de decisão e testes de contrato, contribuiu para uma cobertura mais consistente das regras de negócio e para a redução de riscos associados a falhas funcionais.

A implementação das automações nas camadas de Interface Web (UI), API, Mobile e Performance possibilitou uma visão ampla do comportamento da aplicação sob diferentes perspectivas. A automação de UI e API demonstrou como testes funcionais podem ser estruturados de forma reutilizável e sustentável, enquanto os testes de performance evidenciaram a importância de avaliar o sistema não apenas sob o aspecto funcional, mas também sob carga e concorrência de usuários.

Durante o desenvolvimento do projeto, foram enfrentados desafios técnicos relevantes, especialmente relacionados à diferença de comportamento dos testes entre o ambiente local e o ambiente de Integração Contínua, bem como à instabilidade observada nos testes Mobile. Esses desafios reforçaram a compreensão de que a automação de testes exige constante análise, ajustes e maturidade técnica, além de evidenciar a importância da Integração Contínua como ferramenta para identificar fragilidades que não são perceptíveis apenas em execuções locais.

Como principal aprendizado, destaca-se a importância de uma abordagem de testes bem planejada, que considere não apenas a automação, mas também a qualidade do código de teste, a manutenibilidade, a organização dos artefatos e a confiabilidade das evidências geradas. Além disso, o projeto reforçou a necessidade de transparência técnica, reconhecendo limitações e decisões de escopo como parte natural do processo de desenvolvimento de software.

As lições aprendidas neste trabalho podem ser aplicadas diretamente na prática profissional, especialmente no planejamento de estratégias de testes, na escolha adequada de ferramentas, na implementação de pipelines de Integração Contínua e na análise crítica dos resultados obtidos. Dessa forma, este TCC contribuiu significativamente para o desenvolvimento técnico e profissional na área de Quality Assurance, consolidando conhecimentos e preparando para a atuação em projetos reais de software.

6. REFERÊNCIAS BIBLIOGRÁFICAS

EBAC – Escola Britânica de Artes Criativas e Tecnologia. **Curso Livre de Engenheiro de Qualidade de Software**. Material didático dos módulos do curso. São Paulo: EBAC, 2025.

CYPRESS.IO. **Cypress Documentation**. Disponível em: <https://docs.cypress.io>. Acesso em: 2025.

CUCUMBER. **Cucumber Documentation**. Disponível em: <https://cucumber.io/docs>. Acesso em: 2025.

SUPERTEST. **Biblioteca para testes de APIs HTTP em Node.js**. Disponível em: <https://github.com/ladjs/supertest>. Acesso em: 2025.

APPIUM. **Appium Documentation**. Disponível em: <https://appium.io/docs/en/latest/> . Acesso em: 2025.

WEBDRIVERIO. **WebdriverIO Documentation**. Disponível em: <https://webdriver.io/docs/gettingstarted>. Acesso em: 2025.

GRAFANA LABS. **k6 Documentation**. Disponível em: <https://k6.io/docs>. Acesso em: 2025.

W3C – **World Wide Web Consortium. HTML Living Standard**. Disponível em: <https://html.spec.whatwg.org/>. Acesso em: 2025.

W3C – **World Wide Web Consortium. Cascading Style Sheets (CSS) – Specifications**. Disponível em: <https://www.w3.org/Style/CSS/>. Acesso em: 2025.